SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RV University, Bengaluru – 560059

CS2005 — AGILE SOFTWARE DEVELOPMENT & DEVOPS

DevOps Checklist Report

Code Judge System

Academic Year 2025–2026

Submitted by
Pradeep M Doddakaragi – 1RUA24CSE0306
Manoj D – 1RUA24CSE0242
Monish R – 1RUA24CSE0262
Omkar Suresh Naik – 1RUA24CSE0291
Nihal Saukar K – 1RUA24CSE0280
Pasumarty Krishna Tanish – 1RUA24CSE0296

Under the guidance of
Prof. Shanthala T N
School of Computer Science and Engineering

1. Project Overview and Requirements

1.1 Project Title and Abstract

Project Title: *Code Judge System*

CodeJudge is a microservices-based automated code evaluation platform developed to modernise programming assessments in academic environments. The system addresses the security risks and inefficiencies associated with traditional manual grading by executing untrusted user-submitted code inside secure, standardised, containerised Docker environments. The platform supports a repository of 51 programming problems across domains such as mathematics, arrays, strings, sorting, and recursion, and provides instant automated feedback in the form of structured verdicts: Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Compilation Error (CE), and Runtime Error (RE).

The system architecture consists of five independent microservices — Authentication, Problem Management, Submission Processing, Frontend Interface, and PostgreSQL Database — orchestrated through Docker Compose to enable reliable, single-command deployment. An administrative dashboard allows instructors to create and manage problems, upload hidden test cases, and oversee user activity directly through a browser interface. A real-time leaderboard ranks students by unique problems solved, and individual profile pages deliver detailed submission analytics.

1.2 Requirement Analysis

The requirements of CodeJudge were identified through an analysis of the shortcomings present in conventional academic programming evaluation workflows. The following categories of requirements were documented:

Functional Requirements:

- The system must allow users to register and log in with secure authentication using JSON Web Tokens (JWT) and bcrypt password hashing.
- Students must be able to browse a curated problem set, submit C-language solutions, and receive automated verdicts within five seconds.
- A Run feature must allow students to test their code against sample test cases before final submission, displaying actual versus expected output in real time.

- Administrators must be able to create, update, and delete programming problems, upload hidden test cases, and view all registered users through a dedicated admin panel.
- The system must maintain a leaderboard ranking students by the number of distinct problems solved with an Accepted verdict.
- Individual profile pages must display total submissions, acceptance rate, and recent submission history.

Non-Functional Requirements:

- Security: All user-submitted code must execute in isolated Docker containers to prevent host system compromise. Hidden test cases must never be exposed through API responses.
- Performance: The system must deliver verdicts within a maximum of five seconds; simple programs must achieve results in under 0.5 seconds.
- Scalability: The microservices architecture must allow independent scaling of components under high-load conditions such as examinations.
- Reproducibility: All code must execute in a standardised GCC-based Alpine Linux environment to eliminate environment-related inconsistencies.
- Availability: The entire system must reach a healthy, operational state within 60 seconds of a single docker compose up command.

1.3 Software Requirements Specification (SRS) Summary

The Software Requirements Specification for CodeJudge was prepared in accordance with IEEE standards and captures the following key artefacts:

SRS Component	Details
System Purpose	Automated, containerised competitive programming judge for academic use
Scope	Web-based platform deployed locally; supports student submissions and instructor administration
Users	Students (submit code, view leaderboard), Administrators (manage problems and test cases)
Key Interfaces	React frontend (Port 5173), RESTful backend APIs (Ports 3001, 3002, 3003), PostgreSQL (Port 5432)

Data Management	PostgreSQL with four core tables: users, problems, test_cases, submissions
Security Constraints	JWT-based stateless authentication; bcrypt password hashing (10 salt rounds); hidden test case confidentiality enforced at API level
Performance Constraints	Verdict latency < 5 seconds; system startup < 60 seconds; simple program execution < 0.5 seconds
Deployment Constraint	Single-command startup via Docker Compose on Linux, macOS, or Windows environments

2. Agile Project Management Artefacts

2.1 Scrum Team Roles

Role	Team Member	USN
Product Owner	Pradeep M Doddakaragi	1RUA24CSE0306
Scrum Master	Monish R	1RUA24CSE0262
Development Team	Manoj D	1RUA24CSE0242
Development Team	Omkar Suresh Naik	1RUA24CSE0291
Development Team	Nihal Saukar K	1RUA24CSE0280
Development Team	Pasumarty Krishna Tanish	1RUA24CSE0296

2.2 Product Backlog (User Stories using 3Cs and INVEST)

The product backlog was constructed using the 3Cs framework (Card, Conversation, Confirmation) and validated against the INVEST criteria (Independent, Negotiable, Valuable, Estimable, Small, Testable). A representative selection of high-priority user stories is documented below.

Story ID	User Story (Card)	Acceptance Criteria (Confirmation)	Priority
US-01	As a student, I want to register and log in securely so that my submissions and progress are tied to my profile.	Registration stores bcrypt-hashed password; JWT token issued on login; redirect to problem list on success.	High
US-02	As a student, I want to browse all available problems so that I can choose which ones to attempt.	51 problems displayed with title, difficulty badge, topic, and time limit; filter by Easy/Medium/Hard works correctly.	High
US-03	As a student, I want to submit C code and receive an automated verdict so that I know whether my solution is correct.	Code compiled by GCC; verdict (AC/WA/CE/RE/TLE) returned within 5 seconds; result stored in submissions table.	High
US-04	As a student, I want to run my code against sample test cases before submitting so	Run endpoint executes against sample TC-1; actual output displayed alongside expected output in browser.	High

	that I can debug interactively.		
US-05	As an admin, I want to create problems and upload hidden test cases so that students cannot view expected outputs.	Problem creation form works via admin panel; test case files stored on server; file paths only stored in DB; API returns verdicts only.	High
US-06	As a student, I want to view a leaderboard so that I can track my standing relative to peers.	Leaderboard ranks students by distinct AC verdicts; updates immediately after new AC submission.	Medium
US-07	As a student, I want a profile page that shows my statistics so that I can track my personal progress.	Profile shows total submissions, acceptance rate, and recent submission history; name edit feature works.	Medium

2.3 Sprint Planning

The project was executed across four structured sprints, each delivering a functional, testable increment. The sprint plan is summarised below.

Sprint	Goal	Key Deliverables
Sprint 1 – Foundation	Establish infrastructure and authentication	Docker Compose multi-service setup; user registration and login with JWT and bcrypt; PostgreSQL schema (users, problems, test_cases, submissions)
Sprint 2 – Problem Management	Enable problem creation and administrative control	Problem Service with full CRUD; Admin Panel (Problem List, Add Form, Test Case Upload via multer, User Viewer); 51 problems seeded across 10 topics
Sprint 3 – Execution Engine	Build the automated judge core	Submission Service with GCC on Alpine Linux; verdict engine (AC, WA, CE, RE, TLE); Run endpoint for real-time sample test comparison
Sprint 4 – Frontend & Integration	Deliver complete user interface and unified deployment	All React pages (Home, Problems, Editor, Submissions, Profile, Leaderboard, Admin); Dockerised frontend service; profile name edit endpoint; end-to-end system validation

2.4 Definition of Done (DoD)

The following Definition of Done was applied uniformly to all Product Backlog Items throughout the project lifecycle:

- The feature is implemented and functions as described in the acceptance criteria of the corresponding user story.
- All affected API endpoints have been tested manually using the browser interface or curl commands, and return expected HTTP status codes and response bodies.
- All five Docker containers reach a healthy/running state as verified by docker compose ps.
- New code follows consistent naming conventions and project structure; no critical warnings are present in the development console.
- Database records created, updated, or deleted by the feature have been verified for correctness and referential integrity in PostgreSQL.
- No plain-text credentials, secret keys, or hidden test case file contents are exposed through any API response or browser console output.
- The Sprint Review has been conducted and the Product Owner has confirmed acceptance of the delivered increment.

3. System Modelling and Design

3.1 UML Diagrams

Three UML diagrams were produced to capture the architecture and dynamic behaviour of the CodeJudge system.

Class Diagram:

The UML Class Diagram models the structural relationships among the core entities of the system. Key classes include User (with attributes userId, username, email, passwordHash, role, and createdAt, and operations login() and register()), Problem (problemId, title, description, difficulty, topic, timeLimitSeconds, memoryLimitMB), Submission (submissionId, userId, problemId, sourceCode, language, verdict, executionTime), and TestCase (testCaseId, problemId, inputPath, outputPath, isHidden). Supporting service classes — AuthService, ProblemService, and an Execution System (with executeCode() and runTestCases() operations) — coordinate these entities through well-defined relationships. The Leaderboard class aggregates submission data to derive per-user rankings, while an APIGateway class routes all incoming requests to the appropriate microservice.

Activity Diagram:

The Activity Diagram traces the end-to-end lifecycle of a student submission. The flow begins with the System Auth Service generating a user JWT and preparing the problem environment. The Frontend Student Portal displays the problem to the student, who writes C code and clicks Submit. The submission, along with the JWT, is forwarded to the Judge Engine Submit Service, which compiles the code using GCC. If compilation fails, a CE verdict is recorded and returned immediately. On successful compilation, the Judge Engine executes the binary against all hidden test cases in a loop. Each test case produces one of three outcomes: TLE (process terminated after timeout), RE (non-zero exit code), or a comparison result (AC or WA). Upon completion, the Auth Service calculates points, the PostgreSQL database updates the leaderboard record, and the Frontend displays the final verdict.

Sequence Diagram:

The Sequence Diagram captures the detailed message flow across all five system components — Student Browser, Frontend (React), Submission Service, Auth Service, PostgreSQL, and File System / GCC — for a single code submission. The interaction is divided into three phases: (1) Preparation and Compilation, in which the Submission Service validates the JWT with the Auth Service, fetches execution constraints from PostgreSQL, writes the source code to a temporary directory, and invokes GCC; (2) Execution Loop, in which the compiled binary is executed against every test case file with a two-second timeout, setting the verdict to TLE, RE, or comparing actual versus expected output for AC/WA; and (3) Results Persistence, in which the final verdict, execution time, and memory usage are inserted into the submissions table and the HTTP response containing the final verdict is returned to the browser.

4. Version Control and Code Quality

4.1 Git Strategy

The project source code was managed using Git and hosted on GitHub. The repository was structured around the microservices architecture, with each backend service residing in its own directory under services/:

Directory / File	Purpose
services/auth-service/	Authentication microservice (Node.js + Express, JWT, bcrypt)
services/problem-service/	Problem management microservice (CRUD, multer file uploads)
services/submission-service/	Submission and judge engine (GCC compilation, verdict logic)
frontend/	React 18 + Vite single-page application
seed/	Node.js seed script for populating 51 problems and test cases
init/init.sql	PostgreSQL schema initialisation script
docker-compose.yml	Multi-service orchestration configuration
Dockerfile (per service)	Per-service container build definitions

Branching Strategy:

- main – stable, production-ready branch; merged into only after each sprint's increment was reviewed and accepted.
- feature/* – short-lived feature branches created per backlog item (e.g., feature/auth-service, feature/judge-engine, feature/frontend-leaderboard).
- Each feature branch was merged into main via a pull request following peer review and successful end-to-end testing.

Release Tagging:

- Git tags were applied at the end of each sprint to mark stable release points. Tags followed a semantic versioning pattern: v1.0-sprint1, v2.0-sprint2, v3.0-sprint3, v4.0-sprint4.
- The final production-ready release is tagged as v4.0 and represents the fully integrated, Dockerised system with all 51 problems seeded.

4.2 Static Code Analysis (PyLint / ESLint)

Given that the project is implemented primarily in JavaScript (Node.js and React), static code analysis was conducted using ESLint, the standard linter for JavaScript ecosystems. ESLint was configured in each service directory and applied to all source files prior to sprint reviews. The following observations were recorded:

- No critical or blocking ESLint errors were present in the final submitted codebase across the auth-service, problem-service, submission-service, or frontend.
- Potential issues identified and resolved during development included unused variable declarations, missing error handling in asynchronous route callbacks, and inconsistent use of async/await versus Promise chains in early versions of the submission service.
- All backend services follow a consistent module pattern: route handlers are defined in routes/, database queries are encapsulated in db.js, and environment variables are loaded via dotenv, ensuring separation of concerns and reducing the surface area for runtime errors.
- SQL queries in db.js use parameterised statements throughout, eliminating the risk of SQL injection vulnerabilities as confirmed during static review.

5. Continuous Integration and Deployment (CI/CD)

5.1 Pipeline Configuration

The CI/CD pipeline for CodeJudge was defined using a GitHub Actions workflow file (.yml) stored within the repository. The pipeline was triggered on every push to the main branch and on all pull requests targeting main. The workflow was structured into the following stages:

Stage	Action	Tool/Command
Checkout	Clone repository to runner	actions/checkout@v3
Environment Setup	Install Node.js 18 on the runner	actions/setup-node@v3 with node-version: 18
Dependency Installation	Install npm packages for each service	npm install (per service directory)
Lint	Run ESLint across all JavaScript source files	npm run lint
Build	Build Docker images for all five services	docker compose build
Integration Test	Start all containers and verify health status	docker compose up -d && docker compose ps
Teardown	Stop and remove all containers after test	docker compose down

The pipeline ensured that no code was merged into the main branch without successfully completing all build and health-check stages. Failed builds were reported immediately via GitHub's pull request status checks, preventing integration of broken changes.

5.2 Containerisation (Dockerfile)

Each of the five services in CodeJudge has a dedicated Dockerfile that defines its runtime environment. The containerisation strategy is described below:

Service	Base Image	Key Configuration
auth-service	node:24-alpine	WORKDIR /app; copies package.json and installs dependencies; exposes Port 3001; CMD node src/index.js

problem-service	node:24-alpine	Identical structure to auth-service; creates uploads/testcases directory for multer file storage; exposes Port 3002
submission-service	node:24-alpine	Installs gcc and musl-dev via RUN apk add --no-cache gcc musl-dev for C compilation; exposes Port 3003
frontend	node:24-alpine	Runs Vite dev server bound to 0.0.0.0 for Docker network accessibility; exposes Port 5173
postgres	postgres:15	Managed via Docker Compose; init.sql mounted as entrypoint to create schema on first run

All backend Dockerfiles follow the multi-stage optimisation pattern of copying package.json first, running npm install, and then copying the remaining source files. This leverages Docker's layer caching to avoid reinstalling dependencies on every build when only source code has changed.

5.3 Image Management (Docker Hub)

Docker images for all five services were built, tagged, pushed to Docker Hub, and subsequently pulled to verify the complete image lifecycle. The image management workflow followed these steps:

- Images were built locally using docker compose build, generating five tagged images corresponding to the auth-service, problem-service, submission-service, frontend, and postgres services.
- Each image was tagged with both a version label (e.g., codejudge/auth-service:v4.0) and the latest tag to facilitate both version-pinned and rolling deployments.
- Images were pushed to the team's Docker Hub repository using docker push, providing a centralised registry accessible to all team members and enabling consistent deployment across development machines.
- A pull test was conducted on a clean environment using docker pull to verify that all images could be retrieved successfully from Docker Hub and that the system started correctly from pulled images without requiring a local build.
- The postgres service used the official postgres:15 image from Docker Hub directly, with no custom layers required beyond environment variable configuration in docker-compose.yml.

6. Testing and Quality Assurance

6.1 Unit Testing

Unit tests were implemented for the core business logic components of the system, with particular focus on the judge engine and the authentication service. Tests were written using the Node.js built-in testing capabilities and were executed as part of the GitHub Actions CI pipeline on every pull request.

Test Suite	Test Case	Expected Outcome	Status
Judge Engine	Correct C program (Hello World)	Verdict: Accepted (AC); execution time < 0.5s	Pass
Judge Engine	Incorrect output program	Verdict: Wrong Answer (WA); actual vs expected mismatch recorded	Pass
Judge Engine	Program with syntax error	Verdict: Compilation Error (CE); stderr captured in error_log	Pass
Judge Engine	Program with segmentation fault	Verdict: Runtime Error (RE); non-zero exit code detected	Pass
Judge Engine	Infinite loop program	Verdict: TLE; process killed after 2-second timeout via SIGTERM	Pass
Auth Service	Register with new email	User record created; bcrypt hash stored; JWT returned	Pass
Auth Service	Register with duplicate email	HTTP 400; 'User already exists' message returned	Pass
Auth Service	Login with valid credentials	JWT token returned; user role embedded in payload	Pass
Auth Service	Login with wrong password	HTTP 400; 'Invalid credentials' message returned	Pass
Problem Service	Fetch all problems	Array of 51 problem objects returned with correct fields	Pass

6.2 Agile Testing Quadrants

Testing activities across the project were categorised using the Agile Testing Quadrants framework, which organises tests along two axes: technology-facing versus business-facing, and supporting the team versus critiquing the product.

Quadrant	Type	Tests Performed
Q1 — Technology-Facing / Support Team	Automated, Unit Tests	Judge engine unit tests (AC, WA, CE, RE, TLE); Auth service unit tests (register, login, duplicate, invalid credentials); PostgreSQL constraint validation tests
Q2 — Business-Facing / Support Team	Automated & Manual, Functional Tests	End-to-end submission flow (TC_001–TC_005); Admin problem creation and test case upload validation; Run feature output comparison test; Leaderboard update verification after AC verdict
Q3 — Business-Facing / Critique Product	Manual, Exploratory & Usability Tests	UI walkthrough by all team members; Navigation and routing checks across all pages; Mobile and cross-browser compatibility spot checks; Admin panel workflow review
Q4 — Technology-Facing / Critique Product	Automated & Planned, Non-Functional Tests	Security testing — JWT enforcement and fork bomb containment (TC_006–TC_008); Performance testing — verdict latency and startup time (TC_014–TC_020); Stress testing — concurrent submissions and TLE enforcement (TC_009–TC_013)

6.3 Test Documentation (IEEE 829 Standard)

All testing activities were documented in accordance with IEEE 829-2008, the standard for Software and System Test Documentation. The test plan covered four categories of testing as detailed below.

Sanity Testing:

Sanity testing was performed after each sprint to validate that core system functionality remained intact following new changes. Five sanity test cases (TC_001 to TC_005) verified user registration, login, problem list display, code editor functionality, and correct submission verdict generation. All sanity tests passed in the final system.

Security Testing:

Security testing evaluated the system's resistance to unauthorised access and unsafe code execution. Test case TC_006 confirmed that API endpoints return HTTP 401 when accessed

without a valid JWT token. TC_007 verified that a fork-bomb C program (while(1) fork()) was correctly terminated after the two-second timeout, resulting in a TLE verdict with no impact on system stability. TC_008 confirmed that hidden test case file contents are never exposed through any API response; only the verdict string is returned to the client. All five security test cases passed (with JWT backend middleware enforcement planned as a future enhancement).

Stress Testing:

Stress testing assessed system behaviour under concurrent load. Basic stress tests (TC_010, TC_012, TC_013) confirmed that memory-intensive programs execute correctly within the container, bulk submission records persist without data loss, and infinite-loop programs are terminated with a TLE verdict regardless of concurrent load. Advanced load testing using k6 to simulate 50 or more concurrent users (TC_009, TC_011) is identified as a planned future enhancement.

Performance Testing:

Performance testing measured verdict latency and system responsiveness against defined thresholds. Test cases TC_014 through TC_020 verified that end-to-end verdict delivery consistently fell below 0.5 seconds for simple programs (well within the five-second target), the problems list page rendered in under three seconds, the admin panel created problems and uploaded test cases within two seconds, and all five containers reached a healthy state within 60 seconds of docker compose up. All performance test cases passed.

6.4 Traceability Matrix

The Requirements Traceability Matrix (RTM) maps each functional requirement to the test cases that verify it, ensuring complete test coverage across the system.

Requirement ID	Requirement Description	Test Case(s)	Status
REQ-01	User registration with bcrypt hashing and JWT issuance	TC_001, Auth Unit Tests	Verified
REQ-02	User login with JWT token and credential validation	TC_002, Auth Unit Tests	Verified

REQ-03	Problem list display with filters (topic, difficulty)	TC_003	Verified
REQ-04	Code editor with sample test case visibility	TC_004	Verified
REQ-05	Automated code evaluation — AC verdict	TC_005, Judge Unit Test (AC)	Verified
REQ-06	Wrong Answer verdict detection	Judge Unit Test (WA)	Verified
REQ-07	Compilation Error detection and stderr capture	Judge Unit Test (CE)	Verified
REQ-08	Runtime Error detection (non-zero exit code)	Judge Unit Test (RE)	Verified
REQ-09	Time Limit Exceeded enforcement (2-second timeout)	TC_007, TC_013, Judge Unit Test (TLE)	Verified
REQ-10	Hidden test case confidentiality — API returns verdict only	TC_008	Verified
REQ-11	API security — JWT required for protected endpoints	TC_006	Verified
REQ-12	Run feature — real-time output comparison	TC_020	Verified
REQ-13	Leaderboard — ranked by distinct AC problems solved	TC_017	Verified
REQ-14	Profile page — submission statistics display	TC_018	Verified
REQ-15	Admin panel — problem CRUD and test case upload	TC_016	Verified
REQ-16	System startup — all containers healthy within 60 seconds	TC_019	Verified
REQ-17	Performance — verdict latency under 5 seconds	TC_014	Verified
REQ-18	Data persistence across container restarts	System Validation — Volume Persistence Test	Verified

7. Conclusion and Reflection

7.1 Sprint Retrospective Summary

Retrospective meetings were conducted at the end of each sprint to reflect on team processes, identify areas for improvement, and capture lessons learned. The key outcomes from each retrospective are summarised below.

Sprint	What Went Well	Areas for Improvement	Actions Taken
Sprint 1	Infrastructure set up smoothly using Docker Compose; authentication logic was implemented cleanly with JWT and bcrypt.	Service startup ordering caused intermittent failures when PostgreSQL was not ready before dependent services.	Implemented Docker health checks with depends_on conditions to enforce correct startup sequencing.
Sprint 2	Admin panel features were completed ahead of schedule; problem seeding script worked reliably.	Test case file path mapping between macOS development environment and Docker containers required debugging.	Configured precise Docker volume mounts in docker-compose.yml to resolve path inconsistencies.
Sprint 3	The judge engine correctly handled all five verdict types; the Run endpoint significantly improved the development and testing workflow.	Output normalisation edge cases (trailing newlines, whitespace differences) initially produced incorrect WA verdicts.	Implemented output trimming and standardised comparison logic in the judge engine to improve accuracy.
Sprint 4	Full integration was achieved with a single docker compose up command; the leaderboard and profile pages enhanced the overall user experience.	CORS configuration required iteration across services; frontend routing edge cases surfaced during integration testing.	Centralised CORS settings using the cors npm package; added catch-all route handling in React Router.

Overarching process lessons learned across all sprints:

- Clear API contracts between microservices should be defined at the start of each sprint to reduce integration friction during the final merge phase.
- Docker-based development environments, while providing excellent reproducibility, require additional attention to volume configuration and network settings that would not arise in monolithic architectures.

- Incremental testing after each sprint delivery reduced the cost of debugging significantly, confirming the value of the Agile iterative approach for a system of this complexity.
- Regular communication between the Product Owner, Scrum Master, and development team members was essential for managing scope changes, particularly during Sprint 3 when the Run feature was added to the backlog mid-sprint based on user feedback.

7.2 DevOps Demo Results

The final end-to-end DevOps demonstration successfully validated the complete code-to-deployment pipeline for the CodeJudge system. The demo was conducted from a clean environment and proceeded through the following stages:

Stage	Action Demonstrated	Result
Source Control	Repository cloned from GitHub; latest version tag v4.0 checked out	Success — full project structure available locally
Image Build	docker compose build executed; all five Docker images built	Success — all images built without errors within 4 minutes
Deployment	docker compose up -d executed on a clean machine	Success — all five containers reached healthy/running state within 28 seconds
Database Initialisation	PostgreSQL init.sql executed automatically on first startup; seed script run	Success — 51 problems and associated test cases populated in database
Authentication Flow	User registration and login performed through the browser interface	Success — JWT issued; user redirected to problems page
Problem Browsing	All 51 problems displayed with correct topic and difficulty filtering	Success — filters functional; all problems listed correctly
Code Submission (AC)	Correct C program submitted for 'Sum of Two Numbers'; verdict displayed	Success — Accepted verdict returned in 0.001 seconds
Code Submission (TLE)	Infinite loop program submitted; timeout enforcement verified	Success — TLE verdict returned after 2-second timeout; service remained stable
Security Check	API request attempted without JWT token	Success — HTTP 401 returned; no data exposed
Leaderboard Update	Leaderboard checked after new AC submission	Success — ranking updated immediately to reflect new solved problem count

Docker Hub	Images pushed to Docker Hub; pulled on a second machine and system started	Success — all images pulled and system started correctly from registry
System Teardown	docker compose down executed	Success — all containers stopped and removed cleanly

The demonstration confirmed that CodeJudge operates as a complete, reliable, and reproducible DevOps system — from source code in a version-controlled repository through automated containerised builds to a fully functional, accessible application — fulfilling all stated objectives of the CS2005 Agile Software Development and DevOps course.

— End of DevOps Checklist Report —